

P0394-r4: Hotel Parallelifornia: terminate() for Parallel Algorithms Exception Handling

Project: ISO/IEC JTC1/SC22/WG21

Audience: SG1

Repository: brycelelbach/wg21_p0333_improving_parallel_algorithms_exception_handling

Author: [JF Bastien](#) (Google)

Author: [Bryce Adelstein Lelbach](#) (Berkeley Lab)

Table of Contents

- 1 Background
- 2 SG1 Feedback on P0333
- 3 SG1 Feedback on D0394
- 4 LEWG Feedback on D0394
- 5 LWG Feedback on D0394
- 6 Proposed Wording Change
- 7 Examples
- 8 Acknowledgments

You can throw any time you like, but the exceptions can never leave.

§ 1. Background

P0333r0 Improving Parallel Algorithm Exception Handling states:

The exception handling behavior of parallel algorithms invoked with `par_unseq` (the `parallel_unsequenced_execution_policy`) is inconsistent with the exception handling behavior of the other two execution policies specified in the IS (`seq` AKA `sequential_execution_policy` and `par` AKA `parallel_execution_policy`).

25.2.4 [algorithms.parallel.exception] states that if an element access function exits via an uncaught exception in a parallel algorithm invoked under the `par_unseq` execution policy, `terminate()` will be called. This is inconsistent with the other two policies, which would exit by throwing either the uncaught exception or an `exception_list` containing (at least) the uncaught exception.

§ 2. SG1 Feedback on P0333

P0333r0 proposed addressing this problem by allowing `par_unseq` element access functions to throw exceptions. SG1's discussion in Oulu concludes that throwing exceptions pessimizes code which cannot be proven to not throw, e.g. when invoking opaque functions which aren't marked as `noexcept` (see [Example #2](#)). Invoking `terminate()` greatly simplifies code generation in these cases.

We therefore propose to fix the inconsistency by making all parallel algorithms invoke `terminate()` if element access functions exit via an uncaught exception. This has the added benefit of removing the need for `exception_list`. A parallel algorithm is still allowed to throw `bad_alloc` (if it fails to acquire temporary memory resources for parallel execution), but nothing else may be thrown. There is existing

precedent for calling `terminate()` when an exception escapes from `thread`, `main()` or a transaction in the [Transaction TS](#).

Removing the need for `exception_list` solves outstanding design concerns with `exception_list` which were raised at Jacksonville during the [discussion of P0024 The Parallelism TS Should be Standardized](#). Specifically, there was concern about having an `exception_list` which was not constructible by users. The consensus in LEWG at Jacksonville was to give `exception_list` user-accessible constructors and mutators for C++17.

[D0322r1](#) `exception_list` proposed a possible design for a user-constructible `exception_list`. Designing this `exception_list`, is a difficult task. `exception_list` derives from `exception`, and in a parallel context it could potentially be caught in multiple threads concurrently. Thus, any `exception_list` design would need to be thread-safe. To ensure thread-safety and to maintain consistency with all other standard exceptions, the authors of D0322r1 felt it was necessary for `exception_list` to be immutable. The standard library does not currently have immutable containers; `exception_list` would be the first, and thus would be exploring an entirely new design space. At Oulu, the authors of D0322r1 and LEWG felt that there was not sufficient time before C++17 to decide on a design for immutable containers in the standard library. By removing the need for `exception_list`, it is not necessary for it to be fixed in time for C++17.

Further issues with `exception_list` include:

- The order in which `exception_list` is populated is unspecified, since the algorithms are concurrent and/or parallel.
- Whether implementations forge ahead or abort early is implementation-defined.
- Which NTBS content is expected in `.what()` considering that `exception_list` is generated at runtime, `.what()` cannot allocate, and an `exception_list` may be very larger. If the NTBS is constructed optimistically based on the contents, it may well run out of memory while enumerating the members. Otherwise, the NTBS is unlikely to provide useful information.
- Whether and when nested `exception_lists` are allowed is unclear.
- As discussed in [N4157 Relaxing Packaging Rules for Exceptions Thrown by Parallel Algorithms](#), walking an `exception_list` is not easy. You must recursively walk the list to deal with nested `exception_lists` and unpackage the underlying `exception_ptr`s (see [Example 1](#)).

§ 3. SG1 Feedback on D0394

SG1 reviewed D0394 on Wednesday morning at Oulu. A straw poll was taken about forwarding this to LEWG for C++17:

<i>SF</i>	<i>F</i>	<i>N</i>	<i>A</i>	<i>SA</i>
12	5	1	0	0

§ 4. LEWG Feedback on D0394

LEWG reviewed D0394 on Thursday morning at Oulu. A straw poll was taken about forwarding this to LWG for C++17:

<i>SF</i>	<i>F</i>	<i>N</i>	<i>A</i>	<i>SA</i>
13	1	2	0	0

§ 5. LWG Feedback on D0394

LEWG reviewed D0394 on Thursday evening at Oulu. A straw poll was taken about sending this to plenary for C++17:

<i>SF</i>	<i>F</i>	<i>N</i>	<i>A</i>	<i>SA</i>
7	6	0	0	0

§ 6. Proposed Wording Change

Add the following clause after 15.5.1.1 (1.12) [except.terminate]:

- when execution of the initial function of a thread exits via an exception (30.3.1.2), or
- **when execution of an element access function (25.2.1) of a parallel algorithm exits via an exception (25.2.4), or**

Apply the following changes to 17.6.1.2 [headers] paragraph 2:

The C++ standard library provides **60** ~~61~~ C++ library headers, as shown in Table 14.

In 17.6.1.2 [headers], delete <exception_list> from Table 14.

In 18.1 [support.general], delete the row for exception lists from Table 29.

Delete 18.8.8 [support.exception.list].

Apply the following changes to 25.2.4 [algorithms.parallel.exceptions] paragraph 2:

During the execution of a parallel algorithm, if the invocation of an element access function exits via an uncaught exception, **terminate() is called.** ~~the behavior of the program is determined by the type of execution policy used to invoke the algorithm:~~

- ~~If the execution policy object is of type parallel_vector_execution_policy, terminate() is called.~~
- ~~If the execution policy object is of type sequential_execution_policy or parallel_execution_policy, the execution of the algorithm exits via an exception. The exception will be an exception_list containing all uncaught exceptions thrown during the invocations of element access functions, or optionally the uncaught exception if there was only one. [Note: For example, when for_each is executed sequentially, if an invocation of the user-provided function object throws an exception, for_each can exit via the uncaught exception, or throw an exception_list containing the original exception exception. - end note] [Note: These guarantees imply that, unless the algorithm has failed to allocate memory and exits via bad_alloc, all exceptions thrown during the execution of the algorithm are communicated to the caller. It is unspecified whether an algorithm implementation will "forge ahead" after encountering and capturing a user exception. - end note] [Note: The algorithm may exit via the bad_alloc exception even if one or more user-provided function objects have exited via an exception. For example, this can happen when an algorithm fails to allocate memory while creating or adding elements to the exception_list object. - end note]~~
- ~~If the execution policy object is of any other type, the behavior is implementation-defined.~~

§ 7. Examples

1.) Example of recursively walking an `exception_list`. The ordering of the list is unspecified. Currently there is no standard library facility to help unpack an `exception_list`.

```

void walk(const exception_ptr& e) {
    try {
        rethrow_exception(e);
    } catch (const range_error& r) {
        cout << "found a range error\n";
    } catch (const exception_list& y) {
        for (auto d : y)
            walk(d);
    }
}

void example(Iter first, Iter last, bool (*p)(const Foo&)) {
    try {
        return none_of(par, first, last, p);
    } catch (...) {
        walk(current_exception());
    }
}

```

2.) Example of how opaque calls can force a vectorizing SIMD compiler to pessimize a function by forcing it to prepare for divergence of control flow.

```

struct foo
{
    foo();
    ~foo();
    void bar(); // Won't throw, but not noexcept
    void bar_noexcept() noexcept;
};

void initialize(vector<foo>& v)
{
    #pragma omp simd
    for (auto& e : v)
    {
        foo f;

        // .bar() might exit via exception. If it does, the exception boils
        // up into this context and we have to do stack unwinding before
        // initialize exits via an uncaught exception.
        //
        // This can lead to a divergence of control flow. Multiple SIMD lanes
        // will be executing .bar() concurrently, but only one of them may
        // throw. This requires masking all the way up the call chain within
        // the SIMD region. This is blisteringly expensive, especially on SIMD
        // architectures without full masking support.
        f.bar();

        e = f;
    }
}

void initialize_faster(vector<foo>& v)
{
    #pragma omp simd
    for (auto& e : v)
    {
        foo f;

        // We do not need to worry about .bar() exiting by uncaught exception,
        // so we do not need to prepare for any divergence of control flow.

        f.bar_noexcept();

        e = f;
    }
}

```

§ 8. Acknowledgments

Our thanks to Pablo Halpern, Alisdair Meredith and various members of both SG1 and LEWG for their contributions to this proposal.